

CS 240 - Lab 7

TAs: Sujan & Niral

Spring 2025

Welcome to the 7th lab of this course!! phew

Instructions:

- This lab focuses on implementing the internal mechanisms and kernel functions of a Support Vector Machine (SVM).
- Create a folder on your Desktop with the following directory structure. The folder must contain only the specified file:
`submission_<roll_number>/`
`|-- svm.py`
- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.
- **MAKE SURE YOU RUN THE CORRECT COMMANDS AS SPECIFIED ABOVE**

1 Question 1 : Did ya figure out SVM's ?

1.1 Support Vector Machine Internals

In this section, you will implement the core components of the Sequential Minimal Optimization (SMO) algorithm used to train a Support Vector Machine. The provided `model.py` handles the main training loop, but it relies on your implementations for checking constraints and making predictions.

1.1.1 KKT Condition Violations

The SMO algorithm works by iteratively selecting pairs of Lagrange multipliers (α) and optimizing them. A multiplier is eligible for optimization if it violates the Karush-Kuhn-Tucker (KKT) conditions.

Given the error for the i -th sample $E_i = f(x_i) - y_i$, a sample violates the KKT conditions (within a numerical tolerance, `tol`) if either of the following holds true:

$$(y_i E_i < -\text{tol} \text{ and } \alpha_i < C)$$

or

$$(y_i E_i > \text{tol} \text{ and } \alpha_i > 0)$$

Implement this logic in the `violates_kkt` function.

1.1.2 Decision Function and Bias computation

The decision function computes the continuous raw score for a set of test points. For a given test point x , the score is evaluated against the support vectors (SV):

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

To compute the optimal bias b during training, we average the bias calculated from all "free" support vectors (those where $0 < \alpha_i < C$, within a small tolerance of $1e-5$). For each free support vector x_i :

$$b_i = y_i - \sum_{j \in SV} \alpha_j y_j K(x_j, x_i)$$

The final bias is the mean of all such b_i . If there are no free support vectors, return 0.0. Implement this in `compute_bias`.

1.2 Kernel Functions

In this section, you will implement several kernel functions that allow the SVM to learn non-linear decision boundaries by projecting data into higher-dimensional spaces.

For two matrices of data points X (shape $N \times P$) and Y (shape $M \times P$), you must return the corresponding kernel matrices.

- **Linear Kernel:**

$$K(X, Y) = XY^T$$

- **Polynomial Kernel:**

$$K(X, Y) = (XY^T + \text{coef0})^{\text{degree}}$$

- **RBF Kernel:**

$$K(x_i, y_j) = \exp(-\gamma \|x_i - y_j\|^2)$$

(Hint: Avoid slow loops. Use vectorized squared distance computations).

- **Normalized RBF Kernel:** Standardize the features of the training data to have zero mean and unit variance. Apply the *exact same* scaling transformation to the test data before computing the standard RBF kernel.

1.2.1 Learnable Kernel (Feature Weighting)

The final task is to implement `learnable_kernel`, where the idea is to design a feature mapping parametrised by a vector w . Here ϕ_w represents the feature mapping.

Hint: One possible approach can be to model this ϕ_w to be $w \odot \phi(x)$ for some $\phi(x)$.

The p-normed distance between two samples in this learned feature space is:

$$d_{ij} = \|\phi_w(x_i) - \phi_w(x_j)\|_p^p$$

Hint: you can choose $p = 1$ or 2 , but if you wanna, you can experiment further or even experiment with other notions of distances. Try to use functions for which it's easier to compute gradient or a closed form solution.

We want to learn a feature representation such that there is a clear margin separation for points in different classes. You can define any loss $J(w)$ which can help you achieve this objective.

Hint: You can refer to the page 7 of the lecture (kernel based methods) for a particular loss function (contrastive loss). There can also be other approaches of using cross-entropy etc. You are free to tinker around.

We update the weights w using gradient descent for a given number of iterations:

$$w \leftarrow w - \eta \nabla_w J(w)$$

You can also implement any closed form solution for the weights if you'd like to.

In the end, we need the kernel matrices only, so you're free to use any approach.

Design Hints: To build a robust feature space, consider how you can manipulate d_{ij} based on the labels y_i and y_j . A well-designed objective typically seeks to optimize a relationship between intra-class variance (d_{ij} when $y_i = y_j$) and inter-class variance (d_{ij} when $y_i \neq y_j$).

Be cautious of overfitting or trivial solutions where all weights collapse to zero or explode to infinity. What would help prevent overfitting? hmmm

Implement the iterative learning loop and return the kernel matrices computed using the final weighted features $\tilde{\phi}(x_{train})\tilde{\phi}(x_{test})^T$.

Grading Note for Learnable Kernel: We are going to grade this specific section based on your model's accuracy over a hidden test set. You can assume a test accuracy of over 85% over the current test set to be "good/correct enough", but we will grade the better accuracies to get higher scores. Do your best to experiment and optimize!

Important Instructions

1. Please strictly follow the provided directory structure. Any deviation will result in a grade of 0.
2. All functions in `svm.py` are autograded. Ensure your code is free of syntax errors and runtime exceptions. A single error in a function may cause the autograder to fail for that specific task.
3. Don't hardcode anything anywhere, we will test on other datasets for sanity checks against these practises.

2 Question 2: Feed-Forward Neural Networks (Ungraded)

In this section, you will implement a modular Feed-Forward Neural Network (FFNN) from scratch using only NumPy. The provided `main.py` handles data generation and plotting, but it relies on your implementation of `ffnn.py`.

I strongly suggest you to not use the template code, but code everything from scratch, use the template just as reference if you're stuck

2.1 Forward Propagation and Activations

The forward pass computes the continuous raw score (pre-activation) and the activated output for each layer. For an input matrix X (stored as `activations[0]`), each layer $l = 0, \dots, L - 1$ computes:

$$\begin{aligned} z^{[l]} &= a^{[l]}W^{[l]} + b^{[l]} \\ a^{[l+1]} &= g(z^{[l]}) \end{aligned}$$

where $W^{[l]}$ is the weight matrix, $b^{[l]}$ is the bias vector, and $g(\cdot)$ is the activation function. The final network output is $\hat{y} = a^{[L]}$.

You must implement the following activation functions and their derivatives:

- **ReLU (Rectified Linear Unit):**

$$\begin{aligned} g(z) &= \max(0, z) \\ g'(z) &= \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

- **Sigmoid:**

$$\begin{aligned} \sigma(z) &= \frac{1}{1 + e^{-z}} \\ \sigma'(z) &= \sigma(z)(1 - \sigma(z)) \end{aligned}$$

2.2 Loss Functions

Before optimizing, we must quantify the network's prediction error. Let m be the number of samples, y be the true labels, and $\hat{y} = a^{[L]}$ be the network's final predictions.

- **Mean Squared Error (MSE)** for regression:

$$\mathcal{L} = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

- **Binary Cross-Entropy (BCE)** for binary classification:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

2.3 Backward Propagation

The backward pass uses the chain rule to propagate gradients from the output back through each layer. We track a **delta** δ — the error signal flowing backwards — and use it to compute gradients for the weights and biases at each layer.

Output delta. When using MSE with a linear output, or BCE with a sigmoid output, the output delta simplifies neatly to:

$$\delta = \hat{y} - y$$

Gradients at each layer. Looping backwards through layers $l = L - 1, \dots, 0$, compute:

$$\nabla_{W^{[l]}} \mathcal{L} = \frac{1}{m} a^{[l]\top} \delta$$

$$\nabla_{b^{[l]}} \mathcal{L} = \frac{1}{m} \sum_{i=1}^m \delta_i$$

Backpropagate delta. Pass the error signal to the previous layer:

$$\delta \leftarrow (\delta W^{[l]\top}) \odot g'(z^{[l]})$$

where $\odot$ denotes element-wise multiplication and g' is the derivative of the activation function at layer l .

Gradient descent update. Parameters are updated using learning rate α :

$$W^{[l]} \leftarrow W^{[l]} - \alpha \nabla_{W^{[l]}} \mathcal{L}$$

$$b^{[l]} \leftarrow b^{[l]} - \alpha \nabla_{b^{[l]}} \mathcal{L}$$

2.4 Some tests:

Once your implementation is correct, use the `main.py` script to observe the behavior of your neural network. Run the following experiments by altering the arguments in your model instantiation in `main.py`:

1. **The Necessity of Non-Linearity:** Set `hidden_activation='linear'`. You will observe that the network completely fails to fit the non-linear moons or sine wave. Mathematically, stacking multiple linear layers collapses into a single affine transformation, proving non-linear activations are strictly required.
2. **Vanishing Gradients (Sigmoid vs. ReLU):** In your deep regressor (`layer_sizes=[1, 32, 32, 32, 1]`), change the hidden activation from ReLU to Sigmoid. You will notice training slows down drastically or fails to converge. **Observation Task:** Print out the mean or max of `grad_weights[0]` for the *first* hidden layer during training. Because the derivative of Sigmoid is at most 0.25, multiplying these fractions through multiple layers causes the gradients to exponentially shrink (vanish) to near zero before they reach the early layers.
3. **Symmetry Breaking:** Initialize your weights to exactly zero instead of small random numbers. The network will fail to learn, as every neuron will compute the exact same gradient and update identically.
4. **The Learning Rate:** Train the regressor with very low α (loss barely decreases), huge α (gradients might explode to NaN), and some optimal $\alpha = 0.01$ or $\alpha = 0.1$ (smooth convergence).